Tracking protons through a phantom

Kalman filter vs. global fit, explained step by step

PHY571 Numerical Physics project · team of 2 · 6 weeks · Python (NumPy/SciPy)

The project in one paragraph

We shoot protons (200 MeV) through a block of water (the *phantom*, a fake patient). Inside the water each proton loses energy and is randomly kicked sideways, so its path is not a straight line. Thin silicon planes before and after the phantom record noisy positions (*hits*). A magnet at the end bends the proton, which tells us its exit momentum. We **simulate** all of this ourselves (so we know the truth), then try to **reconstruct** the path and the momentum from the hits only, using two methods: a **global least-squares fit** and a **Kalman filter**. Because we know the truth, we can check exactly how good each method is, and when one beats the other.

Research question. How do multiple scattering and energy loss in the phantom degrade the reconstruction of the proton path and of its exit momentum, and in which regime does a Kalman filter (with energy loss in its model) outperform a global linear fit?

Why it matters. Particle physics: this is exactly how tracks are reconstructed at LHCb, ATLAS and CMS. Medical physics: proton radiography and proton CT (pCT) estimate the *most likely path* of each proton inside a patient with the same mathematics. One code, two fields.

1 The experimental setup

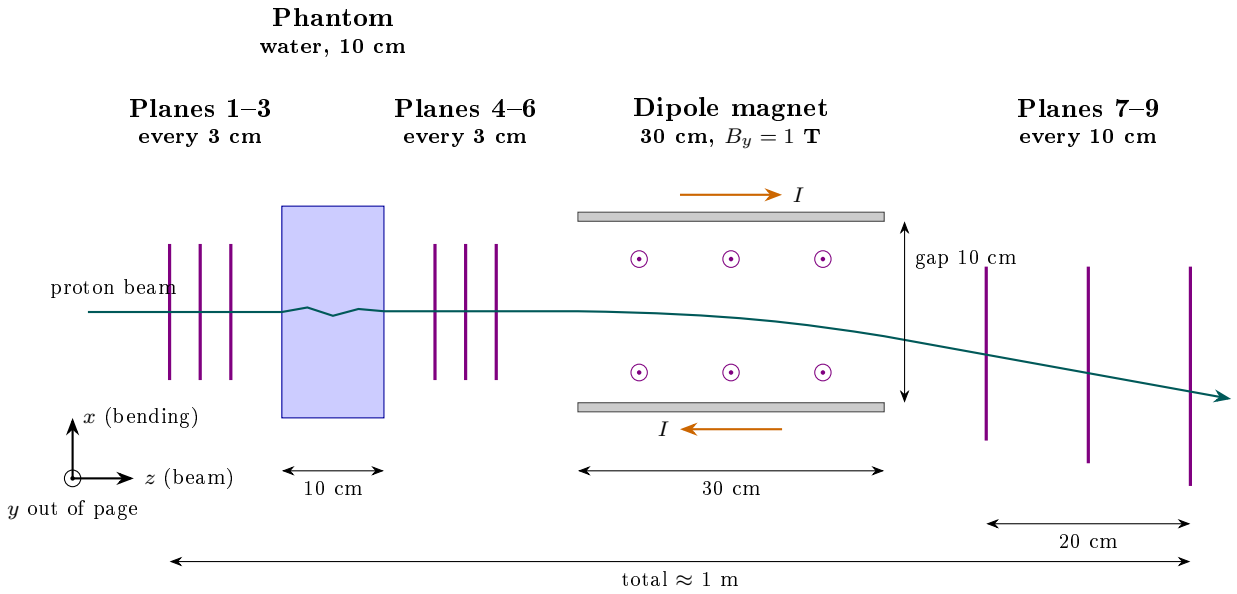


Figure 1: Top view (plane xz) of the toy spectrometer. Transverse scale exaggerated. Violet lines: silicon planes. Circles with a dot: magnetic field B_y pointing out of the page. The magnet is two parallel plates carrying opposite currents I along z. The wiggles inside the phantom represent multiple scattering. Schematic, not a real detector.

1.1 Coordinates

- z: beam direction. We use z as the "time" variable: the proton is followed plane by plane in z.
- x: horizontal, transverse. The magnet bends the proton in x. **This is the plane we simulate.**
- y: vertical, transverse, along the magnetic field. Nothing new happens in y (see §1.4).

1.2 What each part does

Planes 1–3 measure where the proton *enters* and in which direction.
Phantom a block of water, the "patient". The proton loses energy and scatters inside it.
Planes 4–6 measure where the proton *leaves* the phantom and in which direction.
Magnet bends the proton. The bending angle depends on its momentum.
Planes 7–9 measure the direction *after* the magnet. Comparing it with planes 4–6 gives the bending angle, hence the exit momentum. They are spaced more widely (longer lever arm = better angle).

1.3 Default parameters (to be fixed in week 1)

Quantity	Default value	Comment
Beam	protons, $T_0 = 200\,\text{MeV}$, spread $\sim 1\%$	energy known in advance
Silicon planes	9, thickness $300\,\mu\text{m}$, resolution $\sigma \approx 100\,\mu\text{m}$	1D measurement of x
Phantom	water, 10 cm along z	range at 200 MeV $\approx 26\,\text{cm}$
Magnet	30 cm long, gap 10 cm, $B_y = 1\,\text{T}$	two current plates
Gaps	5 cm between groups, 10 cm magnet $\to$ planes 7–9	

Sanity numbers

After 10 cm of water the proton leaves with $T \approx 150\,\text{MeV}$, i.e. $p \approx 0.56\,\text{GeV}/c$. In 1 T the bending radius is $r = p/(0.3B) \approx 1.9\,\text{m}$, so in 30 cm of magnet it turns by $\approx 0.3/1.9 \approx 9°$ and reaches $x \approx 7\,\text{cm}$ at the last plane. It stays inside the 10 cm gap and inside the planes. **If you change energy, phantom or field, redo this check.**

1.4 Why 2D is enough

The field points along y, so the magnet only bends in x. Multiple scattering in x and in y is (for small angles) independent and identical. So y is a statistical copy of x without the interesting part. We work in the xz plane. The track state then has three numbers instead of five:

$$\mathbf{s}(z) = \left(x,\ t_x = \tfrac{dx}{dz},\ q/p\right)^\top, \qquad C \in \mathbb{R}^{3\times 3} \text{ (its covariance).}$$

3D (state $(x, y, t_x, t_y, q/p)$, the one used at LHCb) is an optional extension, interesting mainly for the fringe fields of the magnet.

2 The physics, one effect at a time

2.1 Motion in the magnetic field

A charged particle in a uniform field moves on a circle (in 3D, a helix). With z as independent variable and $\kappa = 0.2998\,\mathrm{GeV\,T^{-1}m^{-1}}$:

$$\frac{dx}{dz} = t_x, \qquad \frac{dt_x}{dz} = -\,\kappa\,\frac{q}{p}\left(1+t_x^2\right)^{3/2} B_y(x,z). \tag{1}$$

In words: the slope changes faster when the field is stronger and when the momentum is smaller. In a uniform field the exact solution is a circle of radius $r[\mathrm{m}] = p[\mathrm{GeV}/c]/(0.3\,B[\mathrm{T}])$: this is our first test of the integrator.

Correction to the previous draft

In 3D the factor is $N_L = \sqrt{1+t_x^2+t_y^2}$ (with a square root). Without it the propagator is wrong, and since the same propagator generates the truth *and* feeds the filter, comparing them would *not* reveal the bug. Only the exact-circle test catches it.

2.2 The magnet: two plates with current

Two parallel plates at $x = \pm d$ carry opposite surface currents K along z. By Ampère's law, between infinite plates the field is uniform, $B_y = \mu_0 K$, and zero outside. For real (finite) plates the field fades smoothly near the entrance and exit (*fringe field*). We model two cases:

1. **Hard-edge field:** $B_y = B_0$ inside, 0 outside. Simple, exact solution.
2. **Biot–Savart field:** compute $\mathbf{B}$ numerically from the finite current sheets (plus their return paths, the circuit must be closed). Check $\nabla\cdot\mathbf{B} = 0$ and $\nabla\times\mathbf{B} = 0$ numerically outside the conductors.

Comparing both tells us whether fringe fields matter for the momentum measurement. (Realistic note: 1 T would need $K \approx 8\times10^5\,\mathrm{A/m}$; real magnets use an iron yoke. Irrelevant for the simulation.)

2.3 Energy loss in water (Bethe–Bloch)

The proton slows down continuously. Its mean energy loss per unit length is

$$-\frac{dE}{dz} = K\frac{Z}{A}\,\rho\,\frac{1}{\beta^2}\left[\frac{1}{2}\ln\frac{2m_e c^2\beta^2\gamma^2 T_{\max}}{I^2} - \beta^2\right], \tag{2}$$

with $K = 0.307\,\mathrm{MeV\,mol^{-1}cm^2}$, $Z/A = 0.555$ and $I = 75\,\mathrm{eV}$ for water. Typical values: 4 MeV/cm to 7 MeV/cm. The loss grows as the proton slows down (Bragg peak at the end of the range). **In code:** a fifth equation integrated together with (1),

$$\frac{d(q/p)}{dz} = -\frac{(q/p)^2}{\beta}\,\frac{dE}{dz}\sqrt{1+t_x^2} \quad (\text{path length } ds = \sqrt{1+t_x^2}\,dz).$$

Simplest fallback: constant dE/dz. Bonus check: integrate until $E = 0$ and compare the range with tabulated values (NIST PSTAR).

2.4 Multiple scattering (Highland)

Many tiny deflections by nuclei add up to a random angle, approximately Gaussian with width

$$\theta_0 = \frac{13.6\,\mathrm{MeV}}{\beta cp}\sqrt{\frac{\ell}{X_0}}\left[1 + 0.038\ln\frac{\ell}{X_0}\right], \tag{3}$$

where ℓ is the path length and X_0 the radiation length: $X_0 = 36.1\,\text{cm}$ for water (so $\ell/X_0 \approx 2.8\%$ per cm), $X_0 = 9.37\,\text{cm}$ for silicon.

- **In the simulation:** cut the phantom into thin slices; in each slice, draw a random kick $\delta t_x \sim \mathcal{N}(0, \theta_0^2(1+t_x^2)^2)$.
- **In the filter:** the same physics enters as the *process noise* Q (the filter knows the track may have been kicked).

Use the same physics in both places

If the simulation and the filter use different scattering or energy-loss models, the pulls will be wrong and you will not know whether the filter or the model is to blame. Person A and Person B cross-check this explicitly.

2.5 The detector

Each plane measures x with Gaussian noise: $m_k = x_{\text{true}}(z_k) + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$. The planes also scatter the proton a little (thin silicon, small but included).

3 From hits back to the track

3.1 The game: truth vs. reconstruction

1. **Simulate (Monte Carlo).** Generate a proton, propagate it through everything with random scattering, record the true state at each plane.
2. **Measure.** Add Gaussian noise to the true x at each plane $\Rightarrow$ 9 hits. Throw away the truth.
3. **Reconstruct.** From the 9 hits only, estimate $(x, t_x, q/p)$ at each plane.
4. **Compare** with the truth. Repeat for 10^4–10^5 protons and look at the statistics.

3.2 The propagator (the engine of everything)

Given the state at plane k, return the state at plane $k+1$ by integrating (1) plus energy loss with **RK4**. It also returns the **Jacobian** $F_k = \partial \mathbf{s}_{k+1}/\partial \mathbf{s}_k$ (how a small change at the start moves the end). Outside the magnet and without energy loss, F is the "drift" matrix

$$F = \begin{pmatrix} 1 & \Delta z & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix},$$

the same as free propagation in paraxial optics. Inside the magnet, compute F numerically (finite differences) or by integrating it alongside the trajectory, and check that both agree.

3.3 Method 1: global least-squares fit

Write all hits as a function of the initial state and minimise

$$\chi^2 = (\mathbf{m} - \mathbf{h}(\mathbf{s}_0))^\top V^{-1} (\mathbf{m} - \mathbf{h}(\mathbf{s}_0)).$$

The catch: scattering makes downstream hits *correlated* (one kick moves all the following hits), so V is a full 9×9 matrix, not diagonal. Inverting it costs $O(N^3)$. Fine for 9 planes, painful for hundreds. The simple version (diagonal V, ignore scattering) is what we call the "naive global fit".

3.4 Method 2: Kalman filter + RTS smoother

Go plane by plane, keeping a best estimate $\mathbf{s}$ and its uncertainty C. At each plane, two steps:

Predict move to the next plane and add the uncertainty due to scattering:

$$\mathbf{s}^{k|k-1} = f(\mathbf{s}^{k-1}), \qquad C^{k|k-1} = F(C^{k-1} + Q)F^{\top}.$$

Update combine the prediction with the measured hit, weighting each by its uncertainty:

$$K = C^{k|k-1}H^{\top}\,(HC^{k|k-1}H^{\top}+\sigma^2)^{-1}, \quad \mathbf{s}^{k} = \mathbf{s}^{k|k-1}+K\,(m_k - H\mathbf{s}^{k|k-1}), \quad C^{k} = (I-KH)\,C^{k|k-1},$$

with $H = (1, 0, 0)$ because we only measure x.

After the last plane, the **RTS smoother** runs backwards so that every plane benefits from *all* hits, not only the earlier ones. Each step only inverts tiny matrices: cost $O(N)$. With the right Q, the Kalman result equals the full global fit.

Starting point: the beam energy

The beam energy is known in advance (accelerator, ∼1% spread). We start the filter with q/p from the beam and an uncertainty matching that spread. Without this, q/p is almost undetermined before the magnet.

3.5 Optional method 3: Bayesian fit with MCMC (Metropolis–Hastings)

Treat the initial state *and* the scattering angles in each phantom slice as unknowns, and sample their posterior with Metropolis–Hastings. It is slow, but it makes no Gaussian assumption. It gives:

- an independent check of the Kalman uncertainties;
- a method that still works with non-Gaussian scattering (Molière tails) or fake hits, where Kalman struggles;
- real numerical work: step size tuning, autocorrelation, convergence diagnostics.

4 Validation: how we know it works

Because we have the Monte Carlo truth, we test both the value *and* the quoted error.

Test	What we expect
Pulls	$(\hat{s} - s_{\text{true}})/\sigma_{\hat{s}}$ is $\mathcal{N}(0,1)$ for x, t_x, q/p. Width > 1: errors underestimated. Offset: bias.
χ^2 of the fit	follows a χ^2 with $N_{\text{hits}} - 3 = 6$ degrees of freedom (mean 6).
Momentum resolution	σ_p/p vs. p: scattering dominates at low p, hit resolution at high p. Compare with Gluckstern's formula.
WEPL resolution	σ_{WEPL} vs. phantom thickness (the medical-physics figure of merit).
Range	integrated Bethe–Bloch range vs. tabulated values.
Negative control	deliberately wrong Q: the pulls must get wider. If they do not, the test is not sensitive.

WEPL (water-equivalent path length) is the thickness of water that would cause the same energy loss. For a pure water phantom it equals the real thickness, so it is a clean check; it becomes interesting with a bone insert.

Unit tests per module

Module	Acceptance test
Field	$\nabla\cdot\mathbf{B}=0$ and $\nabla\times\mathbf{B}=0$ numerically; infinite-plate limit gives $\mu_0 K$.
Propagator	error $\propto h^4$ (halve the step $\Rightarrow$ error /16); exact circle in uniform B; forward then backward returns the start.
Simulation	fixed seed reproduces; hit residual width equals σ; scattering angle width equals Highland.
Kalman	with $B=0$ and no scattering it equals ordinary least squares; innovations are white noise.

Three known traps

1. Scattering happens *at* the previous plane and is then propagated: $F(C+Q)F^{\top}$, not $FCF^{\top}+Q$.
2. The χ^2 uses *predicted* residuals (before update), not smoothed ones.
3. Nonlinearity: in the magnet the linearisation depends on the track. Iterate: refit linearising around the previous solution until it stops changing.

5 Code organisation

One Python package, small modules with one job each, tests next to them, and **one script that regenerates every figure**.

```
phy571-proton-tracking/
|-- README.md              how to install and run; which script makes which figure
|-- requirements.txt       numpy, scipy, matplotlib, pyyaml, pytest
|-- config/
|   `-- default.yaml       ALL parameters (geometry, beam, field, seeds) in one place
|-- src/ptrack/
|   |-- geometry.py        planes, phantom, magnet positions; loaded from the config
|   |-- field.py           M1: hard-edge field, Biot-Savart, interpolation
|   |-- materials.py       Bethe-Bloch, Highland, X0 and I of materials
|   |-- propagator.py      M2: RK4 step, propagate plane->plane, Jacobian
|   |-- simulation.py      M3: generate protons, scatter, lose energy -> true states
|   |-- detector.py        M3: smear true x into hits with resolution sigma
|   |-- fitting/
|   |   |-- global_fit.py  naive and full-covariance least-squares fit
|   |   |-- kalman.py      M4: predict, update, iterate, RTS smoother
|   |   `-- mcmc.py        optional: Metropolis-Hastings fit
|   |-- validation.py      M5: pulls, chi2, resolutions, WEPL, coverage
|   `-- plotting.py        all figure styles in one place
|-- tests/
|   |-- test_field.py        div B = 0, infinite-plate limit
|   |-- test_propagator.py   h^4 convergence, exact circle, forward-backward
|   |-- test_simulation.py   seed reproducibility, Highland width
|   `-- test_kalman.py       B = 0 equals least squares
|-- scripts/
|   |-- run_simulation.py    simulate N protons -> data/sim_*.npz
|   |-- run_reconstruction.py  fit with each method -> data/fit_*.npz
|   `-- make_figures.py      ALL report figures from data/ -> figures/
|-- notebooks/             quick exploration only; nothing the report depends on
|-- data/                  generated, not committed to git
|-- figures/               generated, not committed to git
`-- report/                Overleaf/LaTeX sources of the report and slides
```

Rules of the game.

- Every number lives in `config/default.yaml`. No magic constants in the code. Each output file saves the config and the seed it was made with.
- Code flows one way: `field`, `materials` → `propagator` → `simulation` / `fitting` → `validation`. Nothing imports "backwards".
- The *same* `propagator.py` is used by the simulation and the filter (the simulation adds random kicks on top).
- Run `pytest` before every merge. A module is "done" when its tests pass, not when it runs.
- Units: lengths in mm, momentum in GeV/c, field in T, energy in MeV. Write it at the top of each file.

6 Work plan

6.1 Who does what

	Person A: *transport*	Person B: *estimation*
Owns	`field.py`, `materials.py`, `propagator.py`	`simulation.py`, `detector.py`, `fitting/`
Numerics	ODEs, RK4, convergence, Biot–Savart integration, interpolation	linear algebra, nonlinear filtering, sampling, statistics
Own tests	exact circle, forward-backward, h^4, $\nabla \cdot \mathbf{B}$	$B = 0$ vs. least squares, pulls, χ^2
Reviews	that Q and the simulation use the same scattering/energy-loss physics	that numerical and analytic Jacobians agree

Together: validation, extension, report and presentation. Short weekly meeting; each milestone reviewed by the other person.

6.2 Six weeks (≈ 54 h per person)

Week	Work	Milestone ("done" means)
1	Repo, config, conventions. Straight tracks: no field, no phantom. Linear Kalman and least squares.	Kalman = least squares on straight tracks.
2	RK4 propagator, convergence study, uniform field. Hits with noise. Kalman with analytic Jacobian.	Exact circle reproduced; h^4 plot.
3	Biot–Savart field, numerical Jacobian, iterated filter.	Kalman converges in the realistic field.
4	Phantom: Bethe–Bloch and Highland in simulation and in Q. RTS smoother.	Pulls $\approx \mathcal{N}(0,1)$ with phantom.
5	Full validation: χ^2, σ_p/p, WEPL, global-fit comparison (accuracy and time).	All core figures produced by `make_figures.py`.
6	One extension, report, slides. No new core code.	Reproducible submission.

6.3 Extensions (only if the core is validated)

Prio.	Extension	What it adds	Effort
1	MCMC (Metropolis–Hastings) fit	third method, checks Kalman errors, Lecture 2 used meaningfully	8–12 h
2	Inhomogeneous phantom (water + bone)	WEPL map, closer to real pCT	6–10 h
3	Adaptive RK (RK45 / Runge–Kutta–Nyström)	accuracy vs. number of field evaluations	3–5 h
4	Molière scattering, outliers	non-Gaussian tails, robustness of Kalman vs. MCMC	4–6 h
5	Fokker–Planck with Crank–Nicolson	beam envelope from a PDE, cross-check of the MC (Lecture 3)	5–8 h
6	Full 3D	fringe-field coupling between x and y	6–10 h

6.4 If things go wrong

- Iterated Kalman does not converge ⇒ go back to uniform field and linear filter, then add complexity one step at a time.
- Bethe–Bloch is a pain ⇒ constant dE/dz. The qualitative effect survives.
- Kalman and global fit give the same answer ⇒ that is a result: report the regime where they agree and why.
- Running out of time ⇒ drop extensions, never validation or the report.

7 Expected results

1. A reproducible simulator: modular code, fixed seeds, every figure regenerated by one script.
2. RK4 convergence study (h^4) and hard-edge vs. Biot–Savart field comparison.
3. Correct pulls and χ^2 for x, t_x, q/p.
4. Momentum resolution vs. p, compared with Gluckstern.
5. WEPL resolution vs. phantom thickness.
6. Quantitative Kalman vs. global fit comparison (accuracy and CPU time).
7. One extension, ideally the MCMC fit.

The conclusion is not "Kalman is always better", but a quantitative map of where each method works and where the model breaks down. A simple, deeply validated model beats a large unchecked one.

8 Link with the course

- **Central:** Monte Carlo simulation (Lecture 2); ODE integration with convergence study (Lecture 1 used Numerov, here RK4, same ideas); numerical linear algebra (the filter itself); numerical integration (Biot–Savart, range).
- **Through extensions:** Metropolis–Hastings (Lecture 2) for the MCMC fit; Crank–Nicolson (Lecture 3) for Fokker–Planck.

Course methods are used where they improve the project, never forced in.

References

[1] R. E. Kalman, *A new approach to linear filtering and prediction problems*, J. Basic Eng. 82 (1960) 35.
[2] R. Frühwirth, *Application of Kalman filtering to track and vertex fitting*, NIM A 262 (1987) 444.
[3] A. Strandlie, R. Frühwirth, *Track and vertex reconstruction: from classical to adaptive methods*, Rev. Mod. Phys. 82 (2010) 1419.
[4] R. Frühwirth, A. Strandlie, *Pattern Recognition, Tracking and Vertex Reconstruction in Particle Detectors*, Springer (2021), open access.
[5] H. E. Rauch, F. Tung, C. T. Striebel, *Maximum likelihood estimates of linear dynamic systems*, AIAA J. 3 (1965) 1445.
[6] Particle Data Group, *Passage of particles through matter*, Review of Particle Physics (current edition).
[7] G. R. Lynch, O. I. Dahl, *Approximations to multiple Coulomb scattering*, NIM B 58 (1991) 6.
[8] R. L. Gluckstern, *Uncertainties in track momentum and direction, due to multiple scattering and measurement errors*, NIM 24 (1963) 381.
[9] E. Lund *et al.*, *Track parameter propagation through the application of a new adaptive Runge–Kutta–Nyström method in the ATLAS experiment*, JINST 4 (2009) P04001.
[10] S. Penfold, Y. Censor, *Techniques in iterative proton CT image reconstruction* (2015), arXiv.
[11] W. H. Press *et al.*, *Numerical Recipes*, 3rd ed., CUP (2007), ch. 7 and 17.
[12] NIST PSTAR database: stopping powers and ranges for protons in water.